# Bridging the Enforcement Verification Gap: Detecting Divergence Between Declared and Actual Security Posture in Kubernetes Containerized Environments

Author: Yehia Sobeh
Supervisor: Kirill Saltanov
Department of Cybersecurity
2026

***Abstract*—Container orchestration platforms such as Kubernetes rely on declarative security primitives to enforce security boundaries around containerized workloads. A fundamental assumption underlies this model: that the container runtime faithfully translates declared security intent into kernel-level enforcement. This paper identifies and addresses the *enforcement verification gap*, defined as the absence of tooling that systematically verifies whether declared Kubernetes security contexts are actually enforced by the container runtime. We present k8s-psa (Kubernetes Pod Security Auditor), a DaemonSet-based tool that continuously compares pod security specifications retrieved from the Kubernetes API against the actual runtime state obtained via the container runtime interface. Through controlled experiments on a Minikube cluster with the Docker runtime, we evaluate the tool across seven test scenarios spanning fully hardened, partially secured, privileged, and deliberately misconfigured workloads. Our results reveal two categories of findings. First, privileged containers that declare seccomp system call filtering have that filtering silently stripped by the Docker runtime, constituting a genuine enforcement gap invisible to existing specification-only tools. Second, the Docker runtime metadata interface omits seccomp enforcement status for non-privileged containers that are correctly protected, constituting an observability gap that produces false positives in naive runtime inspection. These findings demonstrate that the enforcement verification gap is real, that it manifests differently across container configurations, and that runtime inspection must account for the semantic differences between runtime metadata and actual kernel enforcement.**

***Index Terms*—Kubernetes, container security, seccomp, runtime verification, enforcement drift, DevSecOps, security context**

## I. Introduction

Modern software systems increasingly adopt microservice architectures deployed as containerized workloads on Kubernetes. This shift introduces security challenges that traditional perimeter-based approaches cannot address: a larger attack surface distributed across potentially hundreds of containers, complex dependency chains with third-party images, and dynamic runtime behavior where containers are continuously created, scaled, and terminated [1], [2].

Kubernetes addresses these challenges through declarative security primitives specified in pod security contexts. Admission controllers such as OPA Gatekeeper [3] and Kyverno [4] intercept pod creation requests and reject non-compliant specifications. Pod Security Admission (PSA), built into Kubernetes since version 1.25 [5], enforces predefined security profiles at the namespace level.

However, these tools share a critical limitation. Every existing tool reads security properties from the Kubernetes API—that is, from the *declared* pod specification. None verify that the container runtime has correctly translated those declarations into kernel-level enforcement. The translation from a Kubernetes pod spec to actual process isolation involves multiple layers: the kubelet translates the pod spec into a Container Runtime Interface (CRI) request, the container runtime generates an Open Container Initiative (OCI) runtime specification, and a low-level runtime configures Linux kernel primitives [6]. Each translation step may add defaults, override values, or fail to apply certain restrictions.

This paper terms the resulting blind spot the *enforcement verification gap* and presents a tool, k8s-psa, designed to detect divergences between declared security intent and actual runtime enforcement.

### A. Research Questions

- **RQ1:** To what extent does the actual security configuration of running containers diverge from the security properties declared in their Kubernetes pod specifications?
- **RQ2:** Can a lightweight, DaemonSet-based tool effectively detect such divergences with acceptable accuracy and performance overhead?
- **RQ3:** What categories of divergence occur in practice, and how should each category be classified from a security perspective?

### B. Contributions

This work makes four contributions: (1) a novel enforcement verification tool that fills a gap identified in the academic literature; (2) experimental evidence of genuine enforcement drift in privileged containers, where Docker silently strips declared seccomp profiles; (3) discovery of an observability gap in the Docker runtime metadata interface; and (4) an

open-source evaluation framework comprising 44 unit tests, 7 deployment-level test cases, and a real-time dashboard.

## II. Background

### A. Kubernetes Security Architecture

Kubernetes enforces workload security through a layered model. The Kubernetes API accepts pod specifications containing `securityContext` fields that define the intended security posture [6]. The kubelet translates these specifications into CRI requests. The container runtime generates an OCI runtime specification [7]—a JSON document describing the exact Linux kernel configuration. A low-level runtime then configures namespaces, cgroups, capabilities, and seccomp filters.

### B. Pod Security Admission and Policy Engines

Pod Security Admission (PSA), stable since Kubernetes v1.25 [5], enforces Pod Security Standards at admission time. It does not monitor running workloads or verify runtime enforcement. OPA Gatekeeper [3] re-evaluates cluster resources every 60 seconds. Kyverno [4] performs background scans every hour. Both read declared pod specifications from the Kubernetes API. Neither queries the container runtime.

### C. Runtime Behavioral Monitoring

Falco [8] instruments the kernel via eBPF to detect anomalous system calls. These tools monitor what containers *do* at the kernel level but do not verify whether the security configuration matches what was declared.

## III. Literature Review

### A. Static Misconfiguration Detection

Rahman et al. [10] conducted a large-scale empirical study analyzing 2,039 open-source Kubernetes manifests, explicitly identifying runtime verification as future work. Malul et al. [11] proposed GenKubeSec for YAML misconfiguration detection using fine-tuned language models; ~~this work remains an unreviewed preprint.~~ Sgan Cohen et al. [12] presented KubeGuard for configuration analysis via runtime logs, focusing on RBAC and NetworkPolicy; ~~this also remains unreviewed.~~

### B. Configuration Drift Detection

***not compared?***

Kubediff [13] compares running Kubernetes state against version-controlled manifests. The Kubernetes blog [14] described admission controllers for container drift detection. Komodor [15] documented configuration drift approaches. None extend below the Kubernetes API to the container runtime layer.

### C. The Identified Gap

Kermabon-Bobinnec et al. [16] demonstrated that temporal inconsistency between orchestration and runtime layers can be exploited to bypass security policies. Shamim et al. [17] concluded that security researchers should investigate dynamic configuration violation detection. Wong et al. [1] and Devi Priya et al. [2] both identified runtime configuration drift detection as under-explored. No existing open-source tool systematically compares Kubernetes API security declarations against container runtime state. This work fills that gap.

## IV. Methodology

### A. System Architecture

k8s-psa operates as a Kubernetes DaemonSet, deploying one pod per cluster node. Each instance performs periodic two-layer security scans.

**Layer 1 (Spec Check)** reads pod specifications from the Kubernetes API and checks them against eight configurable security policies.

**Layer 2 (Drift Detection)**—the novel contribution—compares declared specifications against actual runtime state by: (1) extracting a `SpecSecurityState` from the Kubernetes API; (2) querying the container runtime (`docker inspect` or `crictl inspect`); (3) normalizing both states to account for expected divergences; and (4) comparing the normalized states across seven security dimensions.

The comparison engine classifies each divergence as **CRITICAL** (declared restriction absent at runtime), **WARNING** (undeclared capability present), or **INFO** (runtime is more restrictive than declared).

### B. Capability Normalization

The container runtime adds 14 default Linux capabilities to non-privileged containers [9]. The normalizer computes the expected capability set from the pod specification: starting with these defaults, removing capabilities in the `drop` list, and adding those in the `add` list. Only deviations from this expected set generate findings.

### C. Experimental Environment

Experiments were conducted on a single-node Minikube cluster (v1.35.0) on Ubuntu 22.04, running Kubernetes v1.32.0 with Docker 27.4.1. The scanner interval was 60 seconds. Seven test pods were designed covering the full spectrum of security configurations, with three scan cycles captured per pod.

## V. Results

### A. Detection Results Summary

Table I presents the Layer 2 drift findings for each test pod.

TABLE I
LAYER 2 DRIFT FINDINGS BY TEST POD

| Test Pod | CRIT | WARN | INFO | Match |
|---|---|---|---|---|
| 1. Fully Hardened | 1* | 0 | 0 | §5.3 |
| 2. No Security Ctx | 0 | 0 | 0 | Yes |
| 3. Priv + Seccomp | 1 | 27 | 0 | Yes |
| 4. Priv, No Seccomp | 0 | 27 | 0 | Yes |
| 5. Partial Security | 1* | 0 | 0 | §5.3 |
| 6. No Privesc | 1* | 0 | 0 | §5.3 |
| 7. Host Namespaces | 0 | 0 | 2 | Yes |

*Observability false positives analyzed in Section 5.3.

### B. True Positive: Privileged Seccomp Stripping

The central finding concerns Test Pod 3 (`privileged: true` with `seccompProfile.type: RuntimeDefault`). The Docker runtime applies all 41 Linux capabilities and removes the seccomp filter entirely, even when seccomp is explicitly declared. This behavior is documented in the Moby project [18].

Test Pod 4 serves as the scientific control: same runtime state, no seccomp declaration, zero CRITICAL findings. The 27 WARNING findings are identical between Pods 3 and 4.

### C. Observability Gap

Test Pods 1, 5, and 6 produced unexpected CRITICAL seccomp findings. Investigation revealed these are false positives: when cri-dockerd translates `RuntimeDefault`, it passes no explicit `SecurityOpt` entry to Docker, relying on Docker's implicit default profile [9]. The CIS Docker Benchmark check 5.21 documents this behavior [19]. An empty `SecurityOpt` indicates the default profile is active, not that seccomp is disabled.

### D. Pod 3 vs. Pod 4: Controlled Comparison

TABLE II
CONTROLLED COMPARISON: POD 3 VS. POD 4

| Property | Pod 3 | Pod 4 |
|---|---|---|
| Spec: privileged | true | true |
| Spec: seccompProfile | RuntimeDefault | Not declared |
| Runtime: capabilities | All 41 | All 41 |
| Runtime: seccomp | Unconfined | Unconfined |
| L2 CRITICAL | **1** | **0** |
| L2 WARNING | 27 | 27 |

Both pods have identical runtime states. The only difference is the seccomp declaration. The drift detector correctly identifies this as the enforcement gap.

### E. Performance Characteristics

Each scan cycle completed in approximately 3.1 seconds for 16 pods. The scanner consumed 64 MiB memory and fewer than 100m CPU. The tool includes 44 automated unit tests executing in 0.2 seconds.

## VI. DISCUSSION

### A. The Gap Is Real but Nuanced

The results confirm the enforcement verification gap but reveal two distinct forms: a *genuine enforcement gap* (privileged containers strip declared seccomp) and an *observability gap* (Docker's metadata omits implicit defaults [9]). The enforcement gap creates a false sense of security. The observability gap complicates verification tool development and demonstrates that kernel-level verification via `/proc/<pid>/status` may be necessary.

### B. Comparison with Existing Tools

No existing open-source tool performs the spec-vs-runtime comparison that k8s-psa implements. OPA Gatekeeper [3], Kyverno [4], and similar tools read only the Kubernetes API. Falco [8] monitors kernel behavior without spec comparison. Kubediff [13] compares the API against git manifests. k8s-psa occupies the space between specification-level tools and kernel-level monitors.

### C. Limitations

The experimental environment uses Minikube with Docker; production clusters use containerd. The seccomp verification relies on runtime metadata rather than kernel-level status. The evaluation was conducted on a single-node cluster. The tool uses CLI commands rather than native API libraries.

### D. Future Work

Three directions merit investigation: extending the detector with `/proc/<pid>/status` verification; deploying on containerd-based clusters; and investigating whether mutating admission webhooks create additional enforcement gaps.

## VII. CONCLUSION

This paper identified and addressed the enforcement verification gap in Kubernetes security. Through controlled experiments, we demonstrated that privileged containers declaring seccomp filtering have that filtering silently removed by the Docker runtime—a genuine enforcement failure invisible to all existing specification-only tools. We additionally discovered an observability gap in Docker's metadata interface. The tool, its evaluation framework, and its real-time dashboard are provided as open-source contributions.

## REFERENCES


[1] A. Y. Wong, E. G. Chekole, M. Ochoa, and J. Zhou, "On the security of containers: Threat modeling, attack analysis, and mitigation strategies," *Comput. Secur.*, vol. 128, art. 103140, May 2023, doi: 10.1016/j.cose.2023.103140.

[2] Devi Priya V S, S. C. Sethuraman, and M. K. Khan, "Container security: Precaution levels, mitigation strategies, and research perspectives," *Comput. Secur.*, vol. 135, art. 103490, Dec. 2023, doi: 10.1016/j.cose.2023.103490.

[3] Open Policy Agent, "Gatekeeper audit," 2024. [Online]. Available: https://open-policy-agent.github.io/gatekeeper/website/docs/audit/

[4] Kyverno, "Background scans," 2024. [Online]. Available: https://kyverno.io/docs/policy-reports/background/

[5] Kubernetes, "Kubernetes v1.25: Pod Security Admission controller in stable," Aug. 2022. [Online]. Available: https://kubernetes.io/blog/2022/08/25/pod-security-admission-stable/

[6] Kubernetes, "Restrict a container's syscalls with seccomp," 2024. [Online]. Available: https://kubernetes.io/docs/tutorials/security/seccomp/
[7] Open Container Initiative, "OCI runtime specification," GitHub, 2023. [Online]. Available: https://github.com/opencontainers/runtime-spec/blob/master/config.md
[8] Sysdig, "Container drift detection with Falco," 2024. [Online]. Available: https://www.sysdig.com/blog/container-drift-detection-with-falco
[9] Docker, "Seccomp security profiles for Docker," Docker Docs, 202.. [Online]. Available: https://docs.docker.com/engine/security/seccomp/
[10] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source Kubernetes manifests: An empirical study," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, art. 99, pp. 1–36, May 2023, doi: 10.1145/3579639.
[11] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation," *arXiv preprint arXiv:2405.19954*, May 2024. [Online]. Available: https://arxiv.org/abs/2405.19954
[12] O. Sgan Cohen, E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "KubeGuard: LLM-assisted Kubernetes hardening via configuration files and runtime logs analysis," *arXiv preprint arXiv:2509.04191*, Sep. 2025. [Online]. Available: https://arxiv.org/abs/2509.04191
[13] Weaveworks, "Kubediff," GitHub, 2020. [Online]. Available: https://github.com/weaveworks/kubediff
[14] Kubernetes, "Using admission controllers to detect container drift at runtime," Dec. 2021. [Online]. Available: https://kubernetes.io/blog/2021/12/21/admission-controllers-for-container-drift/
[15] Komodor, "Drift detection in Kubernetes," 2023. [Online]. Available: https://komodor.com/blog/drift-detection-in-kubernetes/
[16] H. Kermabon-Bobinnec *et al.*, "ProSPEC: Proactive security policy enforcement for containers," in *Proc. 12th ACM Conf. Data Appl. Security Privacy (CODASPY '22)*, Baltimore, MD, USA, Apr. 2022, pp. 1–12, doi: 10.1145/3508398.3511515.
[17] S. I. Shamim, H. Hu, and A. Rahman, "Dynamic application security testing for Kubernetes deployment: An experience report from industry," in *Proc. 33rd ACM Int. Conf. Found. Softw. Eng. (FSE '25), Industry Papers Track*, Trondheim, Norway, Jun. 2025, doi: 10.1145/3696630.3728573.
[18] Moby, "Allow for custom seccomp profiles to be used in privileged containers," GitHub, Issue #41321, 2020. [Online]. Available: https://github.com/moby/moby/issues/41321
[19] Z. Morić, V. Dakić, and T. Čavala, "Security hardening and compliance assessment of Kubernetes control plane and workloads," *J. Cybersecur. Priv.*, vol. 5, no. 2, art. 30, Jun. 2025, doi: 10.3390/jcp5020030.
[20] Aqua Security, "kube-bench," GitHub, 2024. [Online]. Available: https://github.com/aquasecurity/kube-bench